

FitBuddy – AI Fitness Plan Generator

Project Executable Files & Evaluation Guide

Purpose: This document converts the documented FitBuddy project into an evaluator-oriented executable-file submission guide, following the uploaded “Project Executable Files” checklist.

Source basis: FitBuddy project documentation supplied by the user. No undocumented hosted URL, credentials, APK, Docker configuration, repository URL, or test-result claims are invented.

1. Project Summary

FitBuddy is a web-based application that uses Google Gemini AI models to generate personalized 7-day workout plans and nutrition/recovery tips from inputs such as name, age, weight, fitness goal, and workout intensity. Users can submit feedback to update their existing plan. The application uses FastAPI, SQLite with SQLAlchemy, and Jinja2 templates.

2. Executable Submission Checklist

S.No	Required Item	FitBuddy Submission Status	Notes
1	Complete source code	Required	Submit all Python files, templates, static assets, and project folders.
2	README / Setup Guide	Included in this PDF	Run steps are documented below.
3	requirements.txt	Required	Documented dependencies include FastAPI, Uvicorn, Jinja2, SQLAlchemy, python
4	Database schema / seed files	Applicable	SQLite + SQLAlchemy are documented; submit the actual DB/schema files if prese
5	.env.example	Required	Provide GOOGLE_API_KEY placeholder; never submit the real API key.
6	Deployed URL	Not documented	No hosted URL is provided in the source; leave blank or mark N/A.
7	APK / executable binary	N/A	FitBuddy is documented as a web application.
8	Dockerfile	Not documented	Do not claim Docker support unless the actual file exists.
9	Test files/results	Not documented	Add actual test files/results if available.
10	Demo video	Not documented	Add link only if an actual demo video exists.

3. Recommended Project File / Folder Structure

```
FitBuddy/  
■■■■ app/  
■ ■■■■ main.py  
■ ■■■■ routes.py  
■ ■■■■ database.py  
■ ■■■■ gemini_generator.py  
■ ■■■■ gemini_flash_generator.py  
■ ■■■■ updated_plan.py  
■■■■ templates/  
■ ■■■■ index.html  
■ ■■■■ result.html  
■ ■■■■ all_users.html  
■■■■ static/  
■ ■■■■ images/  
■■■■ requirements.txt  
■■■■ .env.example  
■■■■ README.md
```

The source explicitly documents the Python modules above, the three Jinja2 templates, and the /app, /templates, and /static/images project organization. If the actual source tree uses different filenames or locations, submit the real tree rather than renaming files solely to match this example.

4. Main Executable Components

Component	Purpose
main.py	Application entry point referenced by the documented Uvicorn command.
routes.py	Core routing and operational layer connecting frontend, Gemini functions, and SQLite database.
gemini_generator.py	generate_workout_gemini(): creates structured 7-day workout plans using Gemini 1.5 Pro.
gemini_flash_generator.py	generate_nutrition_tip_with_flash(): generates concise nutrition/recovery tips using Gemini Flash.
updated_plan.py	update_workout_plan(): revises the original workout plan using user feedback and Gemini 1.5 Pro.
database.py	Stores users and plans using SQLite + SQLAlchemy; includes save/get/update functions documented in the
index.html	Collects user information and workout preferences.
result.html	Displays generated plan, nutrition tip, and feedback form.
all_users.html	Admin view of users and original/updated plans.

5. Requirements / Dependency File

The documented setup installs the following libraries:

- fastapi
- uvicorn
- jinja2
- sqlalchemy
- python-multipart
- google-generativeai

Recommended requirements.txt content based strictly on the documented dependency list:

```
fastapi  
uvicorn  
jinja2  
sqlalchemy  
python-multipart  
google-generativeai
```

6. Environment Configuration

The project documentation specifies the Gemini API key as an environment variable:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

For submission, provide .env.example with a placeholder only. Do not upload a real API key.

7. Local Execution Instructions

Step 1: Install Python and pip.

Step 2: Open a terminal in the FitBuddy project root.

Step 3: Create a virtual environment:

Command: `python -m venv venv`

Step 4: Activate it. Windows: `venv\Scripts\activate` | Linux/macOS: `source venv/bin/activate`

Step 5: Install dependencies:

Command: `pip install -r requirements.txt`

Step 6: Configure the Gemini API key using `GOOGLE_API_KEY`.

Step 7: Start the FastAPI application:

Command: `uvicorn app.main:app --reload`

Step 8: Open the application at `http://127.0.0.1:8000`

Step 9: Open FastAPI interactive documentation at `http://127.0.0.1:8000/docs`

8. Application Routes

Route	Function	Method
/	Home page / input form	GET
/generate-workout	Generate workout + nutrition tip	POST
/submit-feedback	Update workout plan using feedback	POST
/view-all-users	Admin view of users and plans	GET

9. Execution Flow

User Input → FastAPI route → Pydantic validation → Gemini 1.5 Pro workout generation + Gemini Flash nutrition tip → SQLite/SQLAlchemy storage → Jinja2 result page → User feedback → Gemini 1.5 Pro plan update → Updated plan stored and displayed.

10. Evaluator Test Checklist

1. Open the home page and verify the input form loads.
2. Enter name, user ID, age, weight, fitness goal, and intensity.
3. Submit the form and verify a personalized 7-day plan is displayed.
4. Verify a nutrition/recovery tip is displayed.
5. Submit feedback such as adding cardio or rest days.
6. Verify the updated plan is generated and stored.
7. Open `/view-all-users` and verify stored user and plan information is visible.
8. Open `/docs` and verify the FastAPI application is reachable.

11. Deployment / Access Details

Field	Value / Status
Hosted / Deployed URL	Not provided in the FitBuddy source document.
Demo Login Credentials	Not documented.
Platform / Hosting Provider	Not documented; source describes local deployment.
Repository Link	Not documented.
Demo Video Link	Not documented.

12. Known Issues / Limitations to Record

S.No	Issue / Limitation	Status / Workaround
1	Hosted URL is not provided in the source.	Mark N/A for local-only submission or add actual deployed URL.
2	Docker configuration is not documented.	Do not claim Docker support unless Dockerfile/config exists.
3	Automated test files/results are not documented.	Add actual tests/results if available.
4	Demo credentials are not documented.	Leave N/A unless authentication is implemented.

13. Final Submission Package

For the evaluator submission, package the actual executable project directory together with requirements.txt, README.md, .env.example, database/schema files if applicable, tests if available, and any deployment configuration that actually exists. This PDF should accompany the package as the execution and evaluation guide.

Source & Scope Note

This PDF is based on the uploaded FitBuddy project documentation and the uploaded Project Executable Files checklist. Items explicitly marked “not documented” are intentionally left unresolved rather than invented. The recommended folder tree is an evaluator-friendly organization derived from the documented modules/templates; verify it against the actual project files before submission.